

Fundamental Trade-offs For Key/Value Stores

- Resource cost is bounded by a 2D floor in 3D Read/Update/Memory (RUM) trade-off space
- This means ≥ 2 tuning knobs are required (coordinates on the optimal surface)
- Read-Write (RW) tuning changes stored data structure, which is expensive to re-tune
- Read-Memory (RM) tuning is well-supported via caching and filtering (Bloom, etc.)
- Key Insight:** Write-Memory (WM) + RM enables **Dynamic Tuning**, which simplifies tuning and lowers (re-)tuning costs by avoiding long-term penalties for past tuning decisions

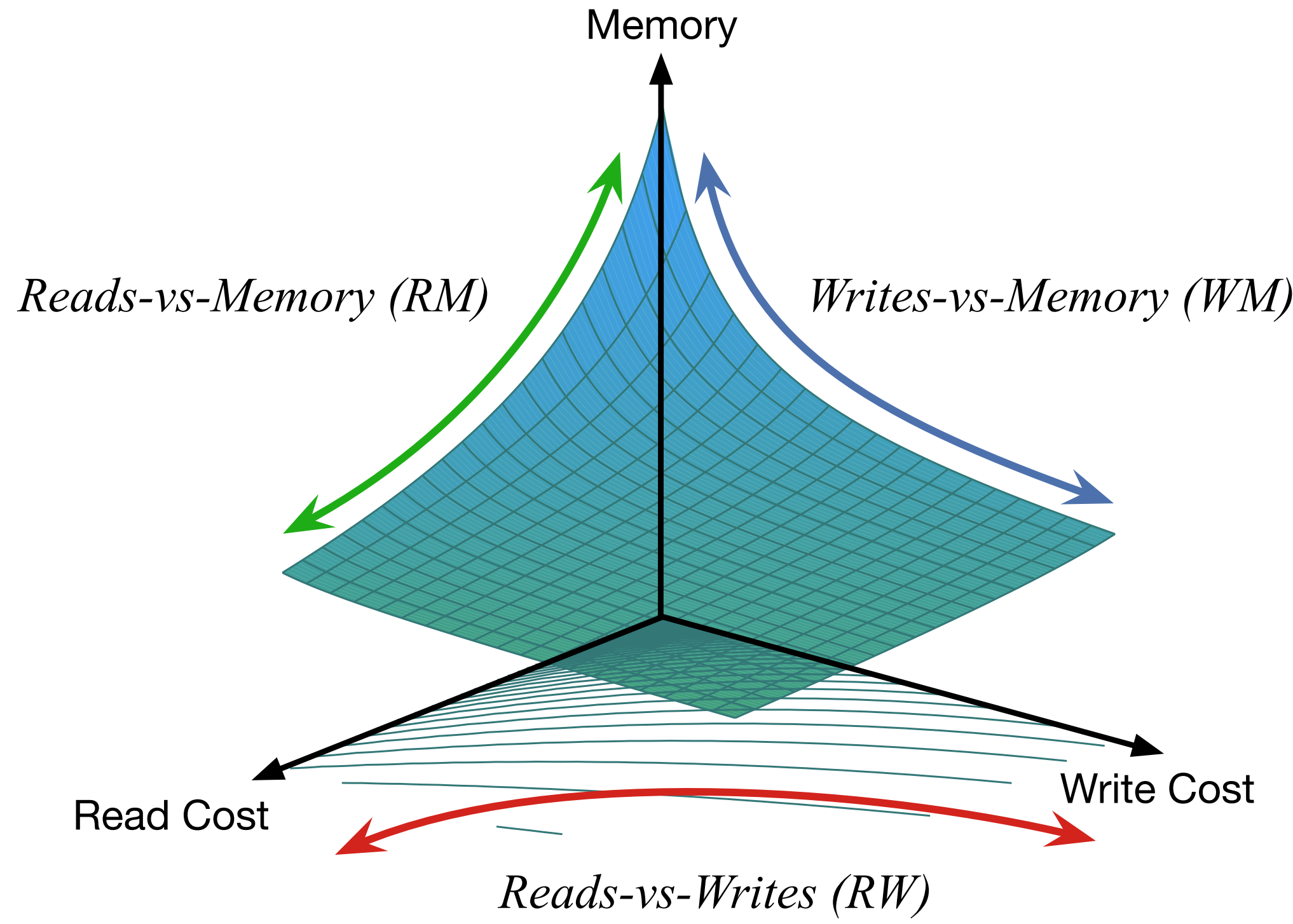


Figure 1. State-of-the-Art key/value stores use data structure selection and compaction policy to trade read efficiency to optimize writes (RW), then fix read performance with caching and filtering (RM). TurtleKV's approach is to use novel data structures and in-memory optimizations to directly trade Reads vs Memory (RM) and Writes vs Memory (WM).

Experimental Results

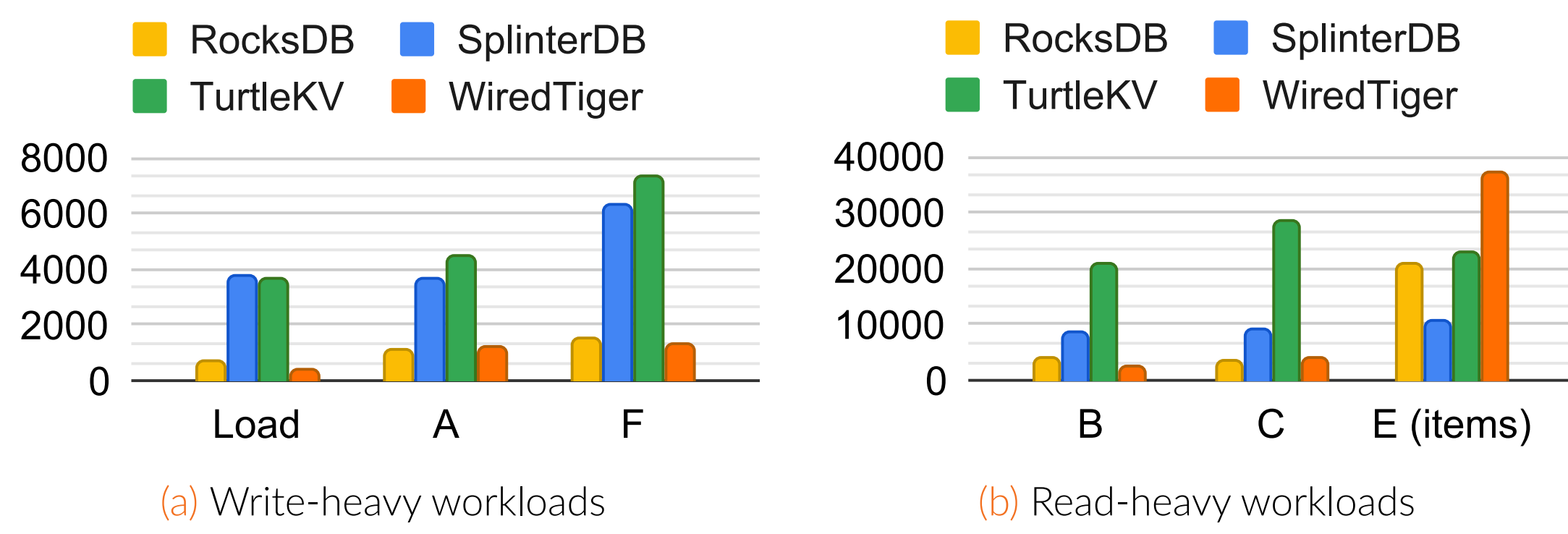


Figure 2. Yahoo Cloud Serving Benchmark (YCSB) 400M×128b (48GiB), t=16, M=48GiB

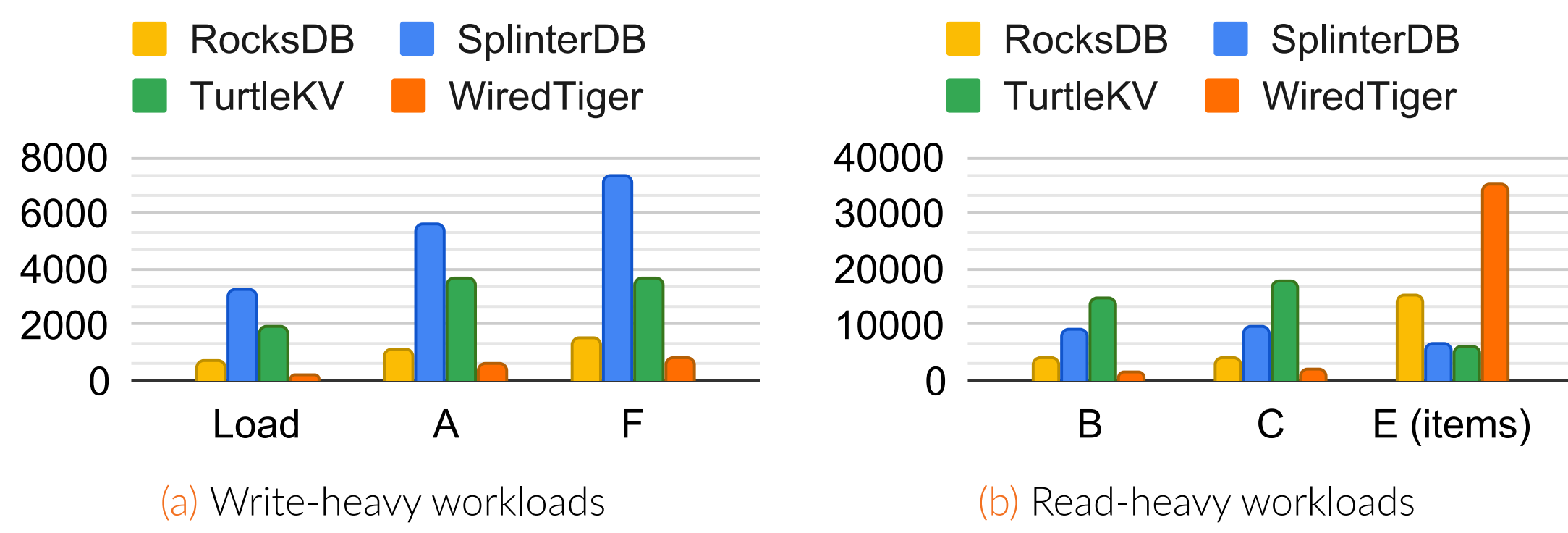


Figure 3. Yahoo Cloud Serving Benchmark (YCSB) 400M×128b (48GiB), t=16, M=16GiB
Load=100% random-order put | A=50% put/50% get | F=50%-m-w/50% get
B=5% put/95% get | C=100% get | E=100% scan (k-items/sec shown)

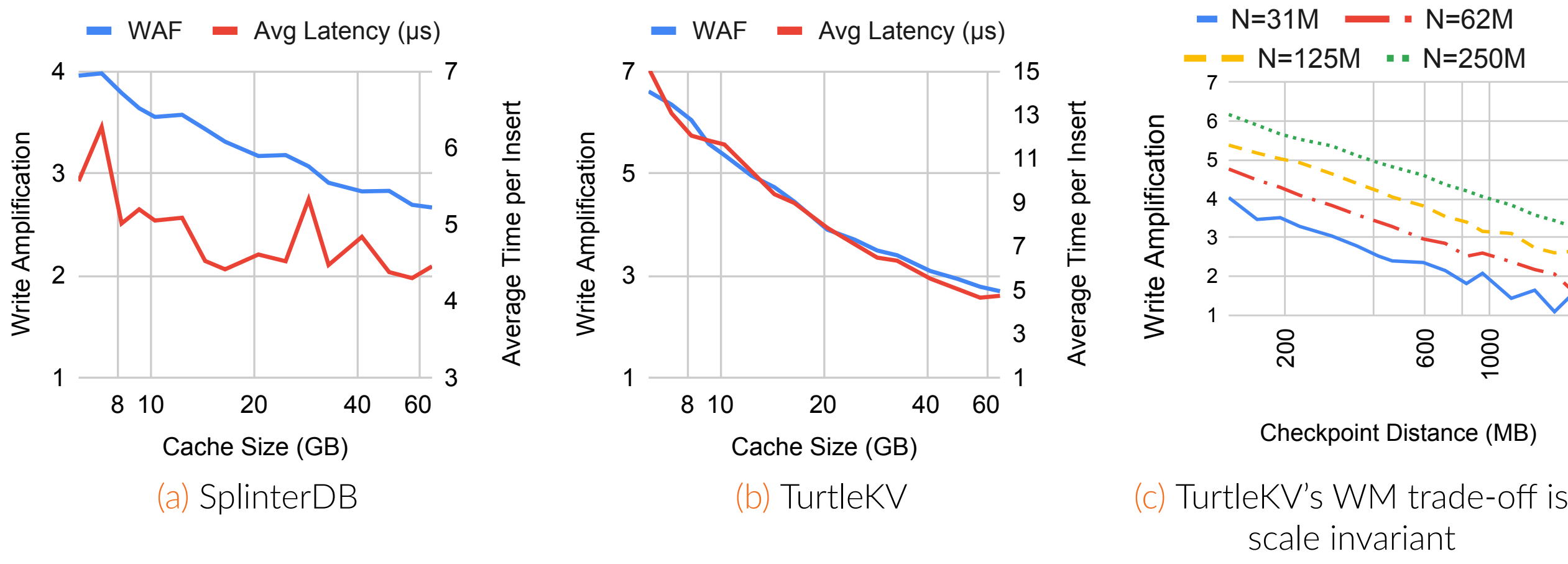


Figure 4. Write Optimization v Cache Size, 500M×128b Load

Experimental Setup. We test TurtleKV against SplinterDB, WiredTiger, and RocksDB using YCSB, on a Linux (6.3, Ubuntu 22) workstation with AMD Threadripper 7970x (32 cores, 64 threads), with 128 GB DDR5-6000 ECC memory and Intel Optane P4800x 1.5TB SSD. Write amplification is measured using `iostat`.

Contact / Learn More



(a) TurtleKV Code (github.com)



(b) TurtleKV Paper (arXiv.org)



(c) Email Author (Tony Astolfi)

Figure 5. Scan QR Codes to Contact the Author and Learn More!

How Well Do Current Key-Value Stores Support Write-vs-Memory (WM) Trade-off Tuning?

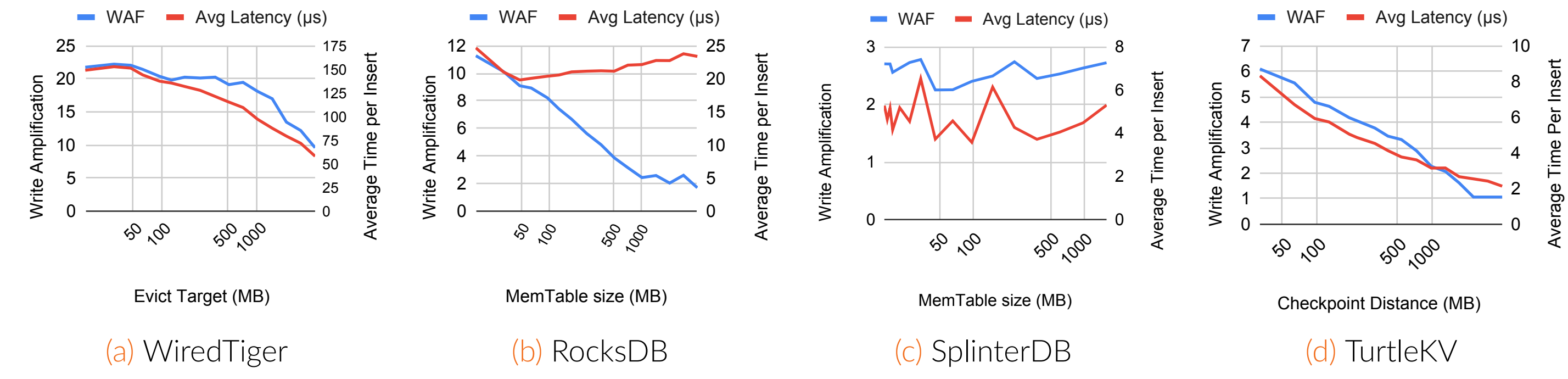


Figure 6. YCSB Load (100M×128b) workload results for various main-memory write buffering (WM) tuning points

	WiredTiger	RocksDB	SplinterDB	TurtleKV
Data Structure supports efficient WM trade-off?	✗	✓	✓	✓
System Design has no in-memory bottlenecks?	✓	✗	✓	✓
First-class write-memory trade-off tuning knob?	✓	✓	✗	✓

Table 1. The three key requirements for dynamic write-memory trade-off tuning in key-value storage engine design.

Key Idea 1: The TurtleTree Data Structure

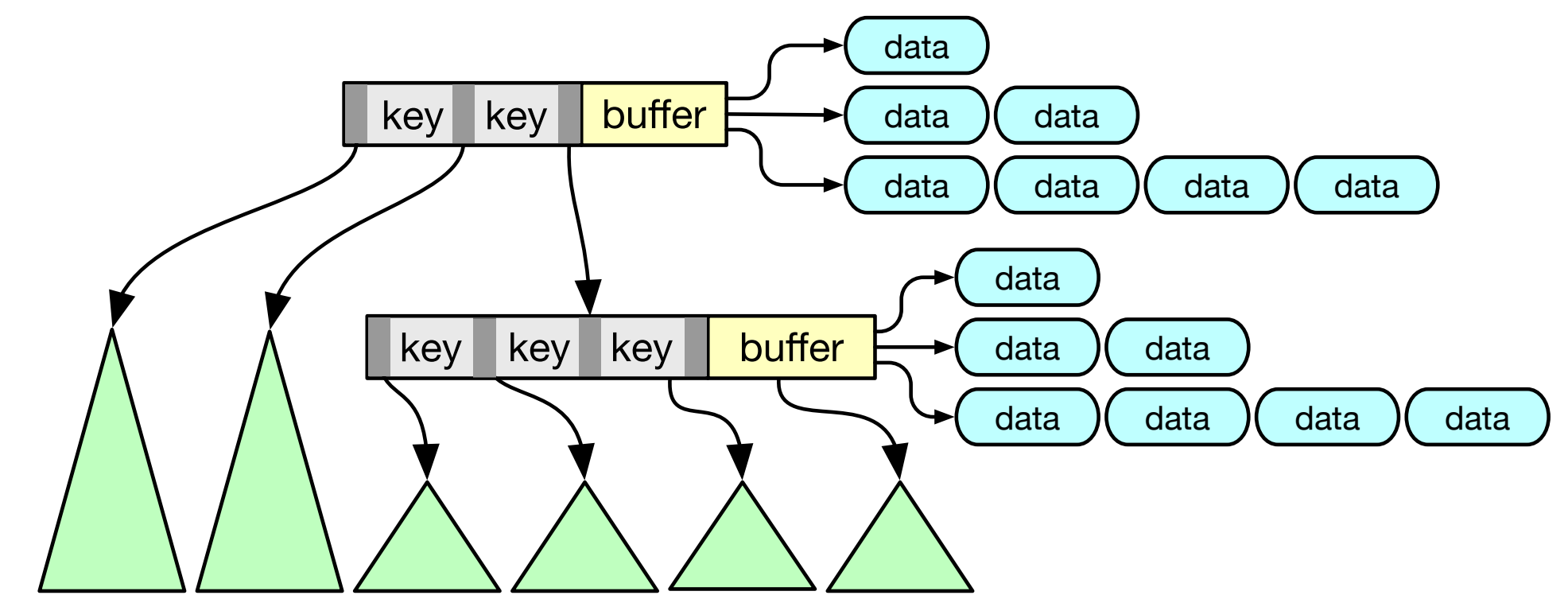


Figure 7. The structure of a TurtleTree, a B⁺-tree variant. Each green triangle is a subtree, which can either be a single leaf page or a recursive copy of the entire structure shown.

- Retain the **trunk-branch separation** of SplinterDB's STB^ε-trees; replace lazy, write-optimized compaction with **read/write-balanced**, scan-friendly policy
- Node buffer pages structured similar to a **fixed-capacity, level-tiered LSM-tree** with fanout 2
- Supports efficient batch update with one-batch-in-one-batch-out (OBI-OBO) flush policy

Key Idea 2: In-Memory Optimized Update Pipeline

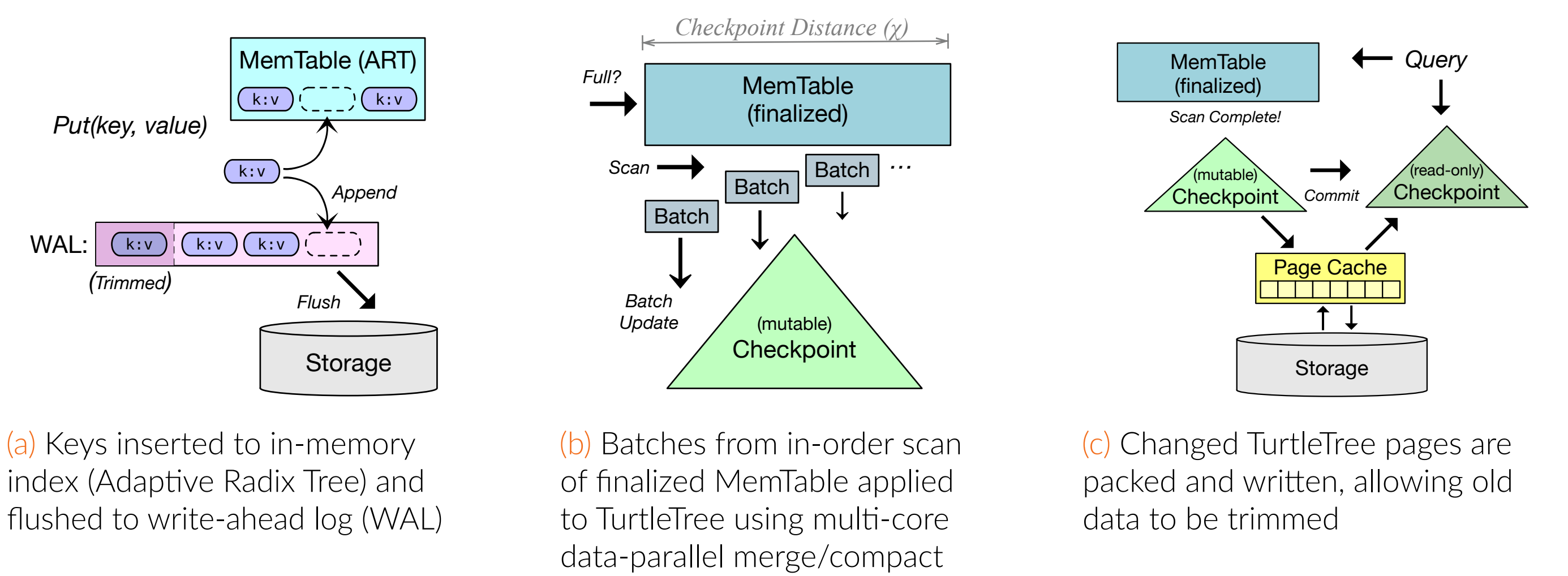


Figure 8. The stages of TurtleKV's update pipeline; bottlenecks are eliminated so write amplification scales along with average update latency over a wide range of tuning points (fig.2d, 8b, 8c)

Key Idea 3: WM Tuning via Checkpoint Distance

- Higher χ requires more memory for write buffering, but reduces write amplification by $\log_2 \chi$
- TurtleKV allows this parameter to be changed dynamically at runtime in response to workload

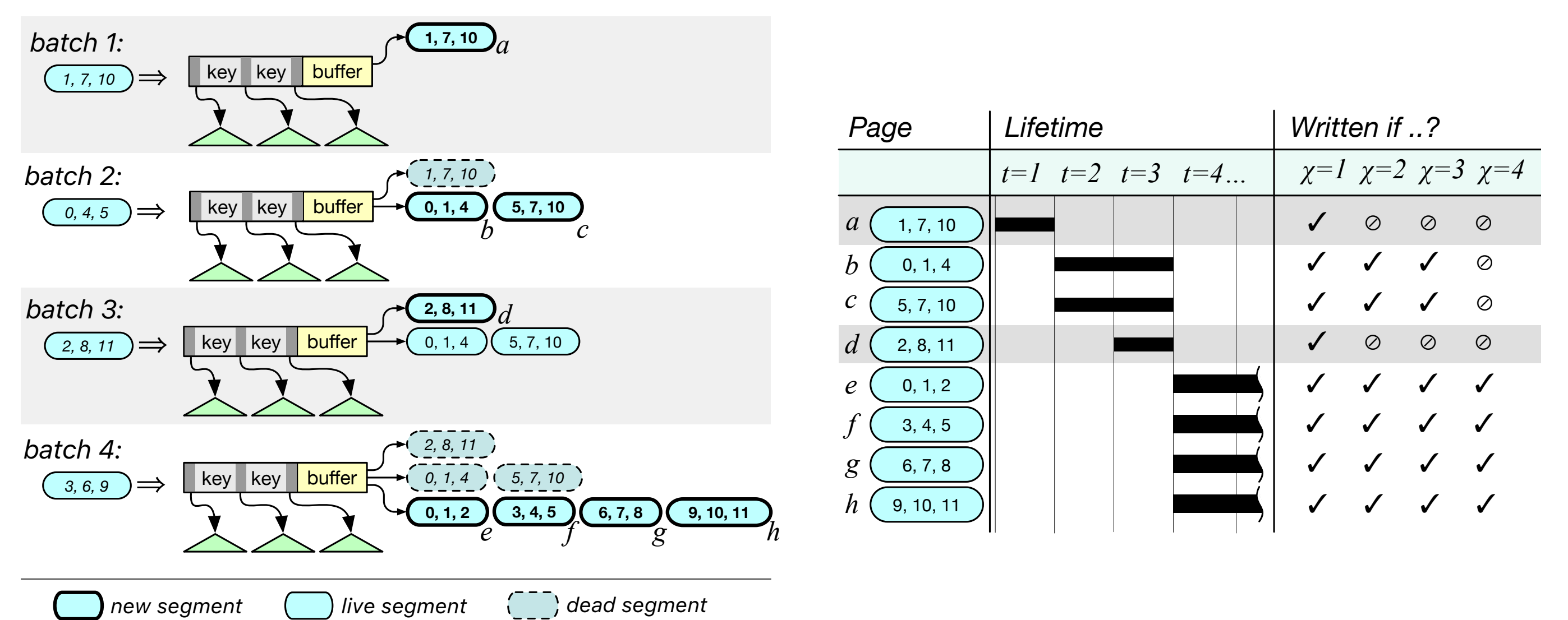


Figure 9. A series of four batch updates to a TurtleTree is shown on the left. On the right, we visualize the lifetime of each buffer segment page. Observe how writing new pages less frequently results in less I/O, because of short-lived buffer segments.